



JÖNKÖPING UNIVERSITY
School of Engineering

Master Thesis

JTH Thesis Template

A quarto template for the modern age of thesis writing

PAPER WITHIN *Industrial Product Realisation*

AUTHOR: *First Author & Second Author*

This final project work has been carried out at the School of Engineering, Jönköping University, in the subject area Industrial Product Realisation. The authors take full responsibility for opinions, conclusions and findings presented.

Examiner:	Name of the examiner
Supervisor:	Name of the supervisor
Scope:	30 credits (second cycle)
Date:	2026-05-XX

Table of contents

Abstract	vi
Preface	vii
List of Abbreviations	viii
Nomenclature	ix
1 Introduction	1
1.1 Background	1
1.2 Problem description	2
1.3 Requirements	2
1.4 Purpose and research questions	2
1.4.1 Rules for good research questions	3
1.4.1.1 The bar	3
1.4.1.2 What counts as engineering work	4
1.4.1.3 What this thesis must not be	5
1.4.1.4 Pointing back to the ILOs	5
1.4.1.5 Bachelor versus master scope	6
1.5 Delimitations	6
1.5.1 Do not delimit away the engineering work	7
1.6 Outline of the thesis	7
2 Theoretical background	8
2.1 Connection between problem and theories	8
2.2 Equations and cross-references	9
2.2.1 Bold math for vectors and matrices	9
2.2.2 Dense matrices and <code>\arraystretch</code>	10
2.2.3 Resizing wide content with <code>\resizebox</code>	11
2.3 Figures and how to reference them	12
2.3.1 Raster versus vector graphics	15
2.3.2 A short detour into typesetting	16
2.3.3 How LaTeX places figures	18
2.3.4 How to influence placement in this template	18
2.4 Citations	19
2.4.1 When to cite	19
2.4.2 What <code>references.bib</code> is	20
2.4.3 How to populate <code>references.bib</code>	20
2.4.4 How to use references in the body	21
2.4.5 What each style produces	21
2.4.6 Why IEEE here does not produce [1]–[3] or [1, 3, 5]	22
2.4.7 Naming the authors in the sentence	22

2.4.8	Switching styles	23
2.4.9	Which style to use	23
2.4.10	Citation, footnote or parenthetical?	24
2.5	Notational style	25
2.5.1	Variables and constants	25
2.5.2	Multiplication	25
2.5.3	Units	25
2.5.4	First-use definitions	26
2.5.5	Common pitfalls	26
3	Methodology	27
3.1	Research design	27
3.2	Data collection	28
3.3	Data analysis	28
3.4	Validity and reliability	28
3.5	Code listings	29
3.6	Executable code and the PDF	31
3.6.1	The pipeline	31
3.6.2	Cell directives	31
3.6.3	Generating figures with matplotlib	32
3.6.4	MechanicsKit	33
3.6.5	Generating equations with sympy	33
3.6.6	Generating tables with pandas	35
3.6.7	Project-wide execution defaults	36
3.6.8	Reproducibility note	36
3.7	Use of artificial intelligence	36
4	Results	38
4.1	What belongs in Results versus Methodology	38
4.1.1	How many concepts	40
4.1.2	Section ordering inside the Results chapter	40
4.2	Presenting CAD figures	40
4.3	Presenting FEA results	41
4.4	Dynamic analysis: energy plots	42
4.5	Verification versus validation	42
5	Analysis	43
5.1	RQ1	43
5.2	RQ2	43
6	Discussion	45
6.1	Theoretical implications	45
6.2	Practical implications	45
6.3	Sustainability and ethics	45
6.4	Methods limitations	46
6.5	Lessons learned	46
7	Conclusions	47
7.1	Requirements summary	47
7.2	Answered research questions	47

7.3 Future work	48
References	49
Appendices	51
Appendix	51
A. Mesh independence study	51
B. Bill of materials	51
C. Test protocol	51
D. Benchmark theses (grade 5 calibration)	51
D.1 Bazzi & Angelou (2018) — Anisotropic polymer simulation	52
D.2 Svensson (2021) — Topology optimisation and additive manufacturing	52
D.3 Frank & Qasemi (2020) — Battery-pack thermal management	52
D.4 Ngo & Lohus (2024) — Constitutive foam modelling for child restraint systems . .	53
D.5 What these four have in common	53

List of Figures

2.1	Overview of which theoretical foundations contributed a solution to the problem within this thesis.	8
2.2	Look at this graph.	13
2.3	Structure of the PID controller.	13
2.4	Stress-strain curve with the 0.2% offset method. Adopted from [9].	14
3.1	PID step response simulation in Python.	30

List of Tables

3.1	PID metrics across ten runs.	35
4.1	Performance comparison of fixed-gain and adaptive PID controllers, averaged across ten runs on a level surface.	38
7.1	Requirements summary.	47

Abstract

The abstract is the second most-read part of the thesis (after the title page). It must work for a wider audience than the thesis itself, who are typically most interested in the results. Write it so it can be read independently from the rest of the report.

A well-structured abstract answers four questions in order:

1. **Purpose and scope.** What is the main message of the thesis, and how far does the investigation reach?
2. **Methods.** What was done to obtain the results (for example, analysed 3 articles, ran 5 experiments, interviewed 10 respondents)?
3. **Results.** What answer was found to the research question? What did the study find?
4. **Conclusion.** What are the larger implications of the findings, especially for the purpose stated in step 1?

Aim for 300 to 500 words. Emphasise the most important elements, use short sentences with varied structure, and keep the vocabulary as plain as possible. Explain or avoid terms and abbreviations that a wider audience will not recognise. Write the abstract last, once the rest of the thesis has stabilised.

Keywords: between three and ten keywords that represent the scope of the work.

Preface

State where and when the thesis work was performed, and which educational programme it concludes. Then acknowledge supervisors, people who helped along the way, family and friends.

This thesis was carried out at the School of Engineering, Jönköping University, as the final project of the *Programme name* programme, between *start month* and *end month*, year.

We would like to thank our supervisor *Name* for guidance throughout the project, our examiner *Name* for valuable feedback, and *Company or department* for providing access to the equipment and data that made the work possible. Finally, we thank our families and friends for their support.

Jönköping, *Month Year*

Author One Author Two

List of Abbreviations

List every abbreviation that appears more than once in the body. Define abbreviations that appear only once inline at first use instead. Keep the list alphabetical.

BLDC Brushless Direct Current (motor)

DE Differential Evolution

FEA Finite Element Analysis

FOC Field-Oriented Control

IMU Inertial Measurement Unit

LQR Linear Quadratic Regulator

MCU Microcontroller Unit

PID Proportional-Integral-Derivative

RQ Research Question

Nomenclature

Optional but recommended once the thesis uses more than a handful of mathematical symbols. List every symbol used in equations, with units in parentheses. Group by Latin then Greek, or keep one alphabetical list.

E	Young's modulus (Pa)
g	Gravitational acceleration (m/s ²)
K_p, K_i, K_d	Proportional, integral, derivative gains of the PID controller
L	Distance from pivot to centre of mass (m)
M_p	Mass of the pendulum body (kg)
r	Wheel radius (m)
$\mathbf{d}$	Nodal displacement vector
$\mathbf{F}$	External nodal force vector
$\mathbf{K}$	Stiffness matrix
α	Gravitational instability coefficient (rad ² /s ²)
ε	Engineering strain
θ	Pitch angle measured from vertical (rad)
σ	Engineering stress (Pa)
σ_y	Yield strength (Pa)

1 Introduction

This chapter introduces the research area, frames the problem, states the purpose and research questions, defines the delimitations, and ends with the outline of the report. It tells the reader why the work matters and what the rest of the thesis sets out to do.

1.1 Background

Provide enough domain context for a reader outside the speciality to understand the problem. Cite the foundational and most recent work that situates the study. Do not write a textbook; cover the field broadly enough that the reader can place the work and trust that the relevant literature has been read.

Example, drawn from a thesis on a two-wheeled self-balancing robot. The first paragraph anchors the field; the second narrows toward the specific class of robot; the third points at the controller problem the thesis will tackle.

The development of robotics has come a long way, especially for mobile robots [1]. Robots that can follow human commands, move autonomously, or run preprogrammed sequences are called mobile robots [2]. Previously, mobile robots were operated by heavy, large and expensive computer systems that were not portable. Modern mobile robots are built with numerous motors and sensors controlled by inexpensive, small and light embedded computers on board.

Within mobile robotics the two-wheeled robot (TWR) is one of the least popular designs because of its structural imbalance and control complexity [3]. To enable self-balancing, TWRs are equipped with inertial measurement units (IMUs) and run advanced control algorithms, typically feedback control, to maintain stability [4].

The classical Proportional-Integral-Derivative (PID) controller has been used on numerous robots, but it is tuned for a single operating point and degrades when external disturbances or parameter variations occur. Adaptive control structures modify their parameters in real time to maintain the desired performance [5], [6], and several have been proposed for stabilising inverted pendulums.

Notice three things in the example. First, every claim that is not common knowledge carries a citation. Second, multiple citations to the same claim use the [a; b] form. Third, sentences are short and concrete, with no filler phrases.

1.2 Problem description

State the concrete problem the thesis addresses. The reader should be able to point at the gap, the bottleneck or the unmet need after reading this section. A common pattern: a paragraph naming the problem, a paragraph on why it matters now, and a paragraph on why existing solutions fall short.

Keep the problem statement narrow enough that the rest of the thesis can plausibly answer it. A useful test: if a reader cannot, from the problem description alone, predict the shape of the research questions, the problem is too broad or too vague.

1.3 Requirements

If the thesis is structured around requirements (typical for engineering projects with an industry partner), list them here. Each requirement should be measurable, with units, so that the results chapter can decide unambiguously whether the requirement is met.

Example of well-formed requirements:

1. The controller shall keep the pitch angle within $|\theta| < 5^\circ$ during steady-state operation on a level surface.
2. The robot shall recover to within $|\theta| < 5^\circ$ within 2 s after an impulse disturbance of $\pm 15^\circ$.
3. The control loop shall run at no less than 200 Hz on the target MCU.
4. Total bill-of-materials cost shall not exceed 1 500 SEK.

Avoid qualitative wording (“the robot shall be stable”, “the system shall be cost-effective”). Translate stakeholder needs into engineering targets with units.

1.4 Purpose and research questions

State the purpose in one or two sentences. Then formulate research questions (RQs) that, taken together, must be answered to fulfil the purpose. Number the RQs. Every RQ must be answered explicitly in the Conclusions chapter.

Example:

The purpose of this thesis is to quantify the control performance and disturbance rejection of a real-time adaptive PID controller, tuned by a bio-inspired optimization algorithm, on a two-wheeled self-balancing robot, in comparison with a conventional fixed-gain PID controller.

RQ1: How do the rise time, settling time, overshoot and steady-state error of the bio-inspired adaptive PID controller compare with those of a Ziegler-Nichols-tuned fixed-gain controller, across an operating envelope spanning 0 to 1 m/s forward velocity?

RQ2: By how much does the adaptive controller reduce the peak tilt angle and the recovery time after a 15° impulse disturbance, compared with the fixed-gain controller, when both run at 200 Hz on the target MCU?

1.4.1 Rules for good research questions

Read this section as a manifesto, not a tip box. The bar laid out here applies to both bachelor and master theses in the *Industrial Product Realisation* and *Mechanical Engineering, Design* programmes at Jönköping University. The School of Engineering also hosts computer science, IT, industrial design, civil engineering, materials and manufacturing, and industrial economy: those programmes set their own bars. The bachelor scope is smaller than the master scope; the rules below are the same.

The Intended Learning Outcomes for this course are listed in your course syllabus. The bachelor and master courses use the same themes with slightly different phrasing and numbering, so this template refers to the themes by name (analysis and evaluation of technical solutions, modelling and simulation, sustainable design, and so on) rather than by ILO number. Map each theme to the numbered ILO in your specific syllabus before you reference it in the body.

1.4.1.1 The bar

This is a thesis in the School of Engineering. The programmes are *Industrial Product Realisation* and *Mechanical Engineering, Design*. The bar is engineering work: simulation, mathematics, computation, physical testing, design-for-manufacture (DfM), design-for-assembly (DfA), tolerance analysis, or some combination thereof. Not just CAD. Not just literature. Not just process description. Not just workshops.

Aim for three research questions. Two is sometimes too thin; four is often a sign that the scope was never sharpened. An RQ that does not demand engineering work is an RQ that does not belong here.

A good RQ passes all four tests:

1. **Specific, not general.** “How can the product be improved?” is vague. Specify what aspect, by what measure, under what conditions.

2. **Not answerable by yes or no alone.** “Does X outperform Y?” or “Can X be simulated?” collapses to yes/no with no engineering depth. Reformulate as “How does X compare to Y, measured by ...” or “How can X be simulated, and what accuracy is achieved against the experimental benchmark?”.
3. **Requires engineering development work.** Simulation, physical testing, prototyping, optimisation, mathematical analysis or DfM/DfA. A question answerable by literature review or by stakeholder workshops alone is insufficient.
4. **Answerable within scope.** If the question needs resources or time beyond the thesis project, narrow it.

A common failure mode is the *overpromised RQ on solid work*: the question asks “what factors cause X” when the answer turns out to be intuitive or already in the textbook. If this happens, reformulate the RQ rather than penalise the work. Either demote the soft work to method (user study, need-finding) and free up the RQ slot for a question with engineering depth, or rewrite the RQ to demand the quantitative analysis the work actually does (“how do X and Y affect Z, and at what threshold does Z fail?”).

1.4.1.2 What counts as engineering work

The thesis must demonstrate at least one of the following in real depth. Most theses combine two or three. A thesis that combines none is not a thesis from these programmes.

- **Simulation and modeling.** FEA, CFD, multi-body dynamics, system simulation, control-loop simulation. Must be validated against measurements or against an analytical benchmark.
- **Mathematical derivation and analysis.** Free-body diagrams, governing equations, parameter studies, sensitivity analysis, dimensional analysis, analytical estimates.
- **Computation.** Algorithms implemented and verified against test cases, scripted parameter sweeps, numerical optimisation, data processing pipelines.
- **Physical testing.** Real measurements on real hardware, with stated equipment make and model, calibration, sample size, statistical treatment, and uncertainty analysis.
- **Manufacturing- and assembly-oriented design.** DfM, DfA, tolerance analysis, manufacturability assessment tied to a specific process (turning, milling, casting, deep drawing, injection moulding, additive manufacturing).
- **Systematic concept selection.** Pugh matrix or weighted decision matrix with explicit, measurable criteria and weighted scores. Not “we picked the one the company liked”.

Every thesis must validate. A product-development thesis that proposes three concepts and stops there is unfinished. A simulation thesis without comparison to measurement or analytical limit is unfinished. A testing thesis with raw data and no analysis is unfinished.

For concrete examples of past theses that meet this bar, see the *Benchmark theses* section in the Appendix. Four grade-5 calibration theses are summarised there with DiVA links.

1.4.1.3 What this thesis must not be

A few patterns recur often enough to warrant naming:

- **The paper product.** Concepts on slides, CAD renderings, glossy mock-ups, no analysis, no measurement, no test. This is a maker hobby with academic formatting. If a high-schooler with SolidWorks could produce the same artefact, the thesis does not fulfil the ILOs on technical-solution analysis and on modelling, simulation or prediction. Build it, simulate it, test it, or cut it.
- **The CAD-only thesis.** A 3D model, some renderings, the words “this could be 3D-printed”. No FEA, no calculations, no manufacturability analysis, no test. CAD is the *output* of engineering thinking, not the substitute for it.
- **The tons-of-text thesis.** A bloated literature review and a chapter on research philosophy (deductive, inductive, qualitative, quantitative, credibility, transferability, dependability, confirmability) bolted onto thin engineering work. Word count is not engineering depth. The reader should learn more about the design and *why it works* than about the methodology of writing about design.
- **The “the company asked us to” thesis.** A sequence of design choices justified by stakeholder preference, not engineering analysis. Stakeholders can be wrong; show why the choice is right, with numbers.
- **The 200-page workshop log.** Interview transcripts, brainstorming output, journey maps. Useful as input. Insufficient as output. The thesis is what was built or analysed *because of* the workshops, not the workshops themselves.

1.4.1.4 Pointing back to the ILOs

The Intended Learning Outcomes are the formal criteria the examiner applies. The themes most often at risk are these. Find the matching numbered ILO in your own course syllabus before you cite one in the body of the thesis.

- **Analysis and evaluation of technical solutions.** Something is *analysed and evaluated* with depth and well-supported arguments. Not just proposed.
- **Planning and execution with adequate methods.** A clear plan, executed with methods appropriate to the question, within the stated time and resource frame.
- **Modelling, simulation, prediction, evaluation.** Claims about how a system behaves are supported by simulation, measurement, or analytical estimate. Not by intuition.

- **Design with sustainability considerations.** Products, processes, or systems are designed with regard to economic, social and ecological sustainability, in concrete terms (mass saved, energy avoided, lifetime extended), not as a list of UN goals.
- **Written and oral communication.** Reproducible, precise, engineering in voice. The reader can rebuild the work from the report.
- **Scientific, societal and ethical judgement.** Reflection on the wider implications of the work, with specific reasoning, not platitudes.

If, after reading the thesis, an external engineer cannot point at the moment where engineering analysis happened, the ILOs on technical-solution analysis and on modelling, simulation or prediction are not met. No amount of process description, literature review, or stakeholder workshop can substitute for that moment. Find it, do it, write it up.

1.4.1.5 Bachelor versus master scope

The scope differs in depth and breadth, not in kind. A bachelor thesis can be a single, sharp investigation: one component, one load case, one measurement campaign. A master thesis is expected to chain two or three of those into a coherent analysis or to push the depth (more material models, more parameter studies, more validation). In both, the rules above apply. A bachelor thesis with a paper product fails the same way a master thesis does.

1.5 Delimitations

*State what the thesis does **not** cover. Delimitations are deliberate scope choices made by the authors, not external constraints. Each delimitation should reduce reader confusion about what to expect from the rest of the report.*

Examples that work well:

- The study is limited by the hardware used in the robot. The computing capacity of the control board, the accuracy and sampling rate of the sensors, and the response speed and torque of the motors are taken as fixed and not optimised further.
- The work focuses only on control and stabilisation. Other aspects of mobile robotics, such as navigation, path planning and obstacle avoidance, are not considered.
- All experiments are performed indoors on a smooth, level surface. Uneven terrain and outdoor environments are not evaluated.

Distinguish delimitations (choices) from limitations (constraints discovered along the way). Limitations belong in the discussion chapter.

1.5.1 Do not delimit away the engineering work

A delimitation is a narrowing of scope, not a way to dodge the engineering bar laid out in section 1.4.1. “We delimit ourselves from performing simulations” is not a delimitation; it is a refusal to write a thesis in these programmes. Same for “we delimit ourselves from testing”, “from mathematical analysis”, “from manufacturability assessment”. The methods are not optional. Narrow *within* the engineering envelope: which load case, which material, which speed range, which manufacturing process, which controller variant. Not which categories of method.

If a delimitation reads like “we choose not to do engineering work on this question”, the question itself is wrong and the right move is to reformulate it (see the *overpromised RQ on solid work* note in section 1.4.1), not to wave the work off in the introduction.

1.6 Outline of the thesis

A short paragraph mapping the rest of the report. Reference each chapter by its label.

The theoretical background in Chapter 2 presents the relevant theory: PID control, inverted pendulum dynamics, and the optimisation algorithms used for adaptive tuning. The methodology in Chapter 3 describes the experimental setup, the data collected and the analysis procedure. Chapter 4 presents the empirical data. Chapter 5 answers the research questions. The report ends with discussion in Chapter 6 and conclusions in Chapter 7.

2 Theoretical background

Present the theory that the rest of the thesis relies on. Cover only what is used later. If a section of theory is not referenced from the method, results or analysis chapters, cut it. Theory is not a literature review; it is the conceptual toolkit the reader needs to follow the argument.

2.1 Connection between problem and theories

A short subsection that maps each theoretical foundation to the part of the problem it helps solve. Helpful both to the reader and as a self-check: if a theory section has no arrow pointing to a part of the problem, it probably does not belong in the thesis.

A simple diagram works well here. In a thesis that combines control theory, embedded software and mechanical design, the figure shows three columns (problem facets, theory blocks, contribution) with arrows connecting them.

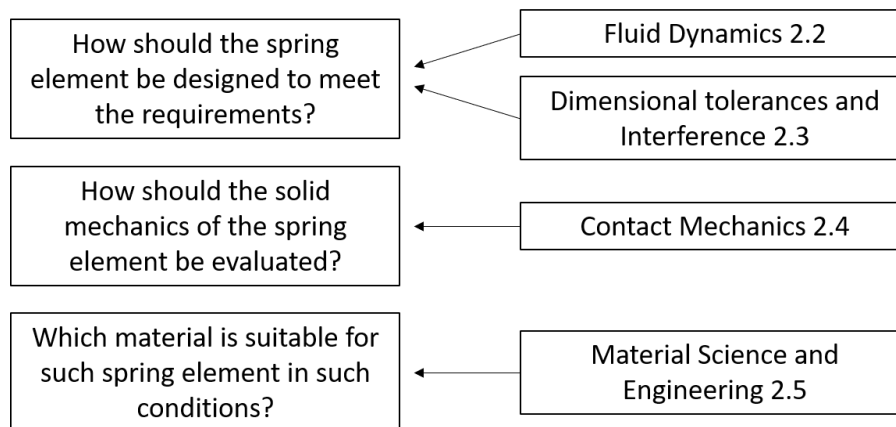


Figure 2.1: Overview of which theoretical foundations contributed a solution to the problem within this thesis.

Reference the figure from the prose, for example: as shown in Figure 2.1, contact mechanics underpins the disassembly-force analysis, while material science feeds the strain-hardening model used in the FEA setup.

2.2 Equations and cross-references

Number every equation that is referenced from prose. Do not number throwaway derivations. Cross-reference with `@eq-label`. Use display math (`$$ ... $$`) for standalone equations and inline math (`$ ... $`) for variables in running text.

Example, taken from fluid dynamics. Bernoulli's equation expresses the energy of an incompressible fluid at a coordinate s : the fluid pressure p_s , the kinetic energy $\frac{1}{2}\rho u_s^2$ and the potential energy $\rho g z_s$ all correlate. In incompressible, frictionless flow the total energy along a streamline is constant, as in Equation 2.1.

$$p_s + \frac{1}{2}\rho u_s^2 + \rho g z_s = \text{constant} \quad (2.1)$$

Applying Equation 2.1 at two points on the same streamline gives Equation 2.2:

$$p_1 + \frac{1}{2}\rho \bar{u}_1^2 + \rho g z_1 = p_2 + \frac{1}{2}\rho \bar{u}_2^2 + \rho g z_2 \quad (2.2)$$

A second example, from controls. The PID control law combines a proportional, integral and derivative response to the error signal $e(t) = r(t) - y(t)$ between reference r and measured output y , following the approach by Hu [7]:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (2.3)$$

Equation 2.3 is used in Chapter 4 to compare the fixed-gain implementation with the adaptive variant.

2.2.1 Bold math for vectors and matrices

Use `\bm{...}` for vectors and matrices. Do not use `\mathbf{...}`; it does not handle Greek letters correctly. Do not put scalars in bold.

Example, drawn from contact mechanics [8]. The potential energy of an elastic body as a function of its nodal displacement vector $\mathbf{d}$ is

$$\Pi(\mathbf{d}) = \frac{1}{2}\mathbf{d}^T \mathbf{K} \mathbf{d} - \mathbf{F}^T \mathbf{d}, \quad (2.4)$$

where $\mathbf{K}$ is the stiffness matrix and $\mathbf{F}$ is the external nodal force vector. Equilibrium minimises Equation 2.4 subject to the kinematic constraint that no node penetrates the rigid obstacle:

$$\begin{cases} \min_{\mathbf{d}} \Pi(\mathbf{d}) \\ \text{s.t. } \mathbf{C}_N \mathbf{d} - \mathbf{g} \leq \mathbf{0} \end{cases} \quad (2.5)$$

Here $\mathbf{C}_N$ is a transformation matrix of the contact normals and $\mathbf{g}$ is the vector of initial gaps. The Karush-Kuhn-Tucker conditions provide the optimal solution:

$$\begin{cases} \mathbf{K}\mathbf{d} - \mathbf{F} + \mathbf{C}_N^T \mathbf{P}_N^A = \mathbf{0} \\ \mathbf{P}_N^A \geq \mathbf{0} \\ \mathbf{C}_N \mathbf{d} - \mathbf{g} \leq \mathbf{0} \\ \mathbf{P}_N^A \circ (\mathbf{C}_N \mathbf{d} - \mathbf{g}) = \mathbf{0} \end{cases} \quad (2.6)$$

2.2.2 Dense matrices and `\arraystretch`

Matrices that contain fractions, integrals or nested subscripts pack their rows so tightly that the result is hard to read. LaTeX exposes a single command to fix it: `\renewcommand{\arraystretch}{factor}`. The factor multiplies the default row height. A value of 1.0 is the default, 1.5 is comfortable for matrices of derivatives, 2.0 is roomy.

Two rules. First, the command must come *before* the matrix, in the same scope, so it is in effect when the rows are typeset. Second, reset it afterwards (back to 1.0) so the relaxed row spacing does not leak into later matrices.

Without `\arraystretch` (default row height; fraction bars in adjacent rows almost touch):

$$\mathbf{J}(\xi, \eta, \zeta) = \begin{bmatrix} \frac{\partial x_h}{\partial \xi} & \frac{\partial y_h}{\partial \xi} & \frac{\partial z_h}{\partial \xi} \\ \frac{\partial x_h}{\partial \eta} & \frac{\partial y_h}{\partial \eta} & \frac{\partial z_h}{\partial \eta} \\ \frac{\partial x_h}{\partial \zeta} & \frac{\partial y_h}{\partial \zeta} & \frac{\partial z_h}{\partial \zeta} \end{bmatrix}$$

With `\arraystretch` set to 1.6, then reset:

$$\mathbf{J}(\xi, \eta, \zeta) = \begin{bmatrix} \frac{\partial x_h}{\partial \xi} & \frac{\partial y_h}{\partial \xi} & \frac{\partial z_h}{\partial \xi} \\ \frac{\partial x_h}{\partial \eta} & \frac{\partial y_h}{\partial \eta} & \frac{\partial z_h}{\partial \eta} \\ \frac{\partial x_h}{\partial \zeta} & \frac{\partial y_h}{\partial \zeta} & \frac{\partial z_h}{\partial \zeta} \end{bmatrix} \quad (2.7)$$

The source for the `with-arraystretch` version is just a `\renewcommand` before the `$$ \dots $$` block and another `\renewcommand` after it. The same trick works for `tabular`, `array` and the various `bmatrix` and `pmatrix` siblings.

One-shot row gap with `\[6pt]`. When only a single row needs breathing space (an integral, a fraction, a block boundary), `\arraystretch` is too blunt: it loosens every row in the matrix. The finer-grained alternative is `\[6pt]` instead of a plain `\` at the end of that one row. The bracketed value is a vertical skip in standard TeX units: `pt`, `ex`, `em`, `mm`.

Example. A block matrix where the upper 2x2 block should be visually separated from the lower block:

$$\mathbf{M} = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \quad (2.8)$$

Source for that matrix:

```

$$
\bm{M} = \begin{bmatrix}
m_{11} & m_{12} \\
m_{21} & m_{22} \\
\bm{0} & \bm{I}
\end{bmatrix}
\end{bmatrix}
$$

```

The `\[10pt]` after row 2 adds 10 pt of vertical space between row 2 and row 3, while rows 1 and 2 stay at the default spacing. Compare with the same matrix using plain `\` everywhere: the boundary between the two blocks disappears and the matrix reads as a single 3x2 grid.

2.2.3 Resizing wide content with `\resizebox`

`\arraystretch` solves the cramped-rows problem. The orthogonal problem is content that overflows the page margin horizontally: a wide matrix, a wide table, a tikz diagram. LaTeX provides `\resizebox{width}{height}{content}` to scale arbitrary content to a target size. Pass `!` for either dimension to preserve aspect ratio. Typical use:

```
\resizebox{\textwidth}{!}{...content...}
```

This scales the content to exactly the text width and lets the height scale proportionally.

Quarto equivalent. Quarto’s `width=` attribute on figures (`![] (path){width=70%}`) handles figure scaling natively. There is no Quarto-native equivalent for scaling a *table* or an *equation*; for those, drop into raw LaTeX. Inside a Quarto markdown file or notebook, raw LaTeX goes in a fenced block tagged `=latex`:

```
```{=latex}
\resizebox{\textwidth}{!}{$
\bm{J}(\xi,\eta,\zeta) = \begin{bmatrix}
\dfrac{\partial x_h}{\partial \xi} & \dfrac{\partial y_h}{\partial \xi} & \\
\hookrightarrow \dfrac{\partial z_h}{\partial \xi} & & \\
\dfrac{\partial x_h}{\partial \eta} & \dfrac{\partial y_h}{\partial \eta} & \\
\hookrightarrow \dfrac{\partial z_h}{\partial \eta} & & \\
\dfrac{\partial x_h}{\partial \zeta} & \dfrac{\partial y_h}{\partial \zeta} & \\
\hookrightarrow \dfrac{\partial z_h}{\partial \zeta} & &
\end{bmatrix}$}
```
```

Use sparingly. Shrinking text below ~8 pt is unreadable; an over-shrunk equation in body text looks out of place. If the content is fundamentally too wide for the page, consider breaking it into two display equations, splitting the table, or rotating the page (`\begin{landscape}...\end{landscape}` from the `pdflscape` package).

2.3 Figures and how to reference them

Every figure needs a caption, a label and a reference from the prose. The reader should not encounter a figure they have not been told to look at.

Syntax: `![caption] (path){#fig-label width=...}`. Reference from prose as `@fig-label`.

The minimal example, one line of markdown:



Figure 2.2: Look at this graph.

As shown in Figure 2.2, one line of markdown becomes a captioned, auto-numbered figure with a clickable cross-reference. The `width=55%` attribute scales the image to 55 % of the text width; common alternatives are `width=4in` (absolute), `width=0.6\textwidth` (raw LaTeX inside the curly braces), or `width=100%` (full text width). Quarto handles aspect ratio automatically when only one dimension is given.

Two more realistic examples follow.

The PID controller is structured as shown in Figure 2.3; the error signal flows into three parallel branches whose outputs are summed into the control signal.

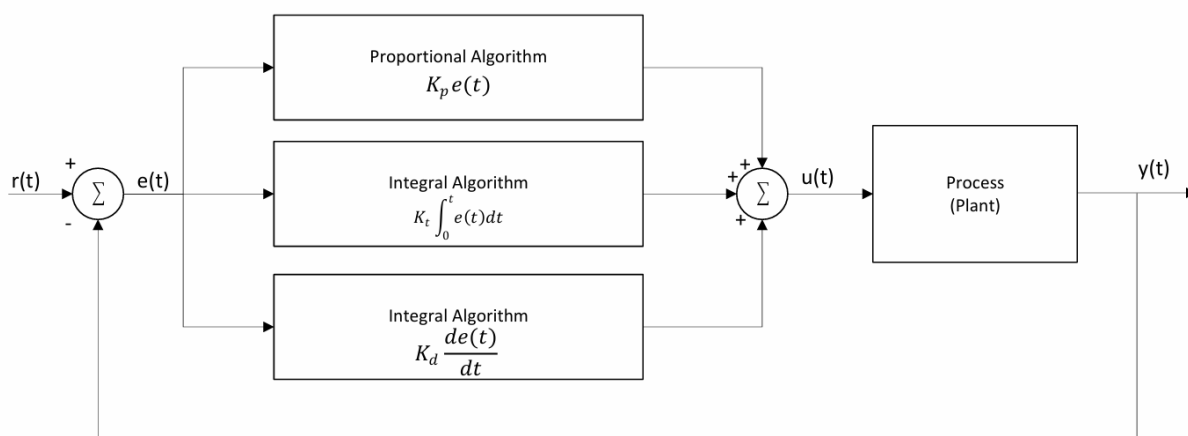


Figure 2.3: Structure of the PID controller.

The stress-strain curve in Figure 2.4 illustrates the offset method used to define the yield strength σ_y ,

since the proportional limit is difficult to locate by inspection [9].

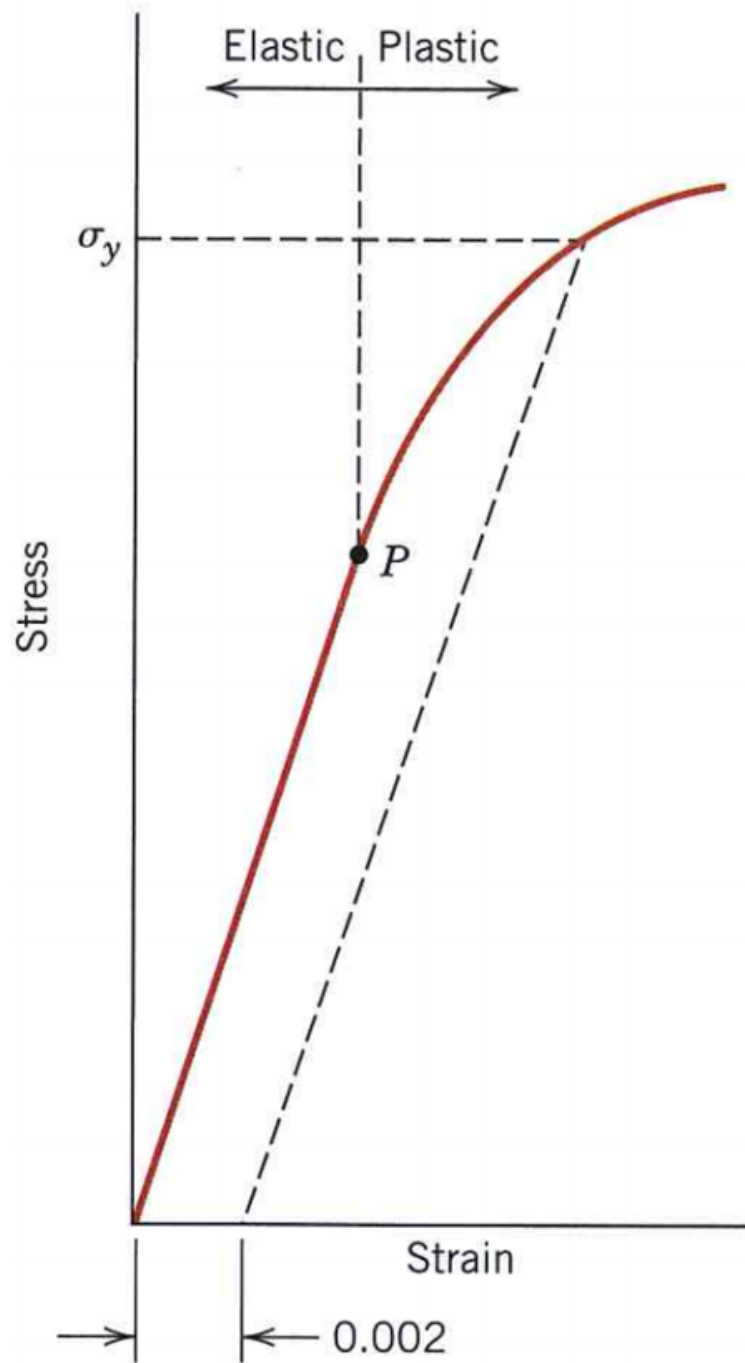


Figure 2.4: Stress-strain curve with the 0.2% offset method. Adopted from [9].

Use SI units in axis labels and captions.

2.3.1 Raster versus vector graphics

When you embed a figure, you choose between two fundamentally different image representations. The choice has more impact on the final PDF than most students realise.

A **raster** image is a grid of pixels. Each pixel has a fixed colour. Zoom past the native resolution and you see the grid: stairsteps on diagonal lines, blocky text labels, blurred curves. Photographs and screenshots are raster. Common file extensions: `.png`, `.jpg`, `.bmp`, `.tiff`.

A **vector** image is a list of geometric instructions: “draw a curve from this point to that point, fill this polygon with this colour, place this text string at this position”. The renderer turns those instructions into pixels at whatever resolution the output device asks for, every time the document is opened. There is no native resolution; the image stays crisp at every zoom level. Common file extensions: `.pdf`, `.svg`, `.eps`.

For a thesis, vector is almost always the right choice. The PDF is read on screens at variable zoom levels, on phones, and printed at 300 to 600 dpi. A pixelated axis label in figure 4.3 distracts every reader, every time. A vector label stays crisp forever, and the source can be edited months later (relabel, recolour, change font) without re-exporting the whole image.

The main case where raster is unavoidable is actual photographs: test rigs, prototypes, fracture surfaces, microscope images. Capture those at high resolution (1500 to 3000 px on the long edge) and embed them as `.png` (lossless) or `.jpg` (lossy, fine for photos). For everything else (CAD-derived diagrams, block diagrams, flow charts, plots, schematics, free-body diagrams) prefer vector.

Tools that produce SVG or PDF:

- **Inkscape** (free, open source). A dedicated vector editor. Save As → PDF or Plain SVG. Both formats embed directly in Quarto.
- **PowerPoint**. Build the diagram with native shapes, arrows and text boxes. Select the diagram, right-click, *Save as Picture...*, choose **SVG**. Works for anything assembled from native PowerPoint primitives. Do *not* paste a raster image into PowerPoint and then export: the export will keep the embedded pixels.
- **draw.io** / **diagrams.net** (free, web-based). Good for block diagrams and flow charts. File → Export As → PDF or SVG.
- **Matplotlib** (Python). Call `plt.savefig("name.pdf")` in the cell; Quarto embeds the vector PDF directly. This is what the executable-code section in Section 3.6 does.
- **TikZ** (LaTeX). Code-based vector graphics that compile alongside the rest of the document. Steeper learning curve, but the result integrates with the body’s fonts and colours by default and stays in version control as plain text.
- **CAD systems**. Most CAD tools can export a drawing view to PDF or DXF. Open the PDF in Inkscape to trim and relabel before embedding.

Most engineering diagrams in a thesis can be drawn in one of these tools in under thirty minutes; the result will look professional and stay editable.

File-size note. Vector files for a typical engineering diagram are tens to a few hundred kilobytes, smaller than the raster equivalent at any reasonable print resolution. The exception is a vector file with thousands of small primitives (a dense plot exported as SVG, or a CAD drawing with every fillet rendered) which can balloon. In that case, export from the source tool with a coarser line tolerance, or rasterise the figure deliberately at 300 dpi.

2.3.2 A short detour into typesetting

To understand why figures sometimes appear “in the wrong place”, it helps to know what is producing the PDF.

Typesetting is the craft of arranging characters, words, lines and floating elements (figures, tables) on a page so the result is legible and pleasant to read. Done well, the reader notices nothing: the eye flows from line to line, the figure appears just where it is needed, the heading has the right amount of space around it. Done poorly, the reader notices everything: orphan lines at the top of a page, a figure half a page after it was mentioned, ragged spacing, a heading hanging at the bottom of a column.

For most of the twentieth century this craft lived with skilled human compositors. In the late 1970s, Donald Knuth, frustrated by the poor typesetting of mathematics in the second edition of *The Art of Computer Programming*, set out to solve the problem in software. The result was **TeX** (1978), a typesetting engine designed around line-breaking and math layout, with algorithms that are still the state of the art four decades later. Knuth froze TeX in 1989; its version number now asymptotically approaches π .

Writing TeX directly is slow because it operates at the level of boxes and glue, not chapters and sections. In 1984 Leslie Lamport built **LaTeX** as a macro layer on top of TeX, providing high-level commands such as `\section{}`, `\figure`, `\cite{}` so authors could describe a document by its structure and let TeX handle the typography. Modern LaTeX is maintained by the LaTeX Project Team and ships in distributions such as TeX Live, MiKTeX and TinyTeX. The actual engine used to compile a document is usually one of pdfTeX, XeTeX or LuaTeX. This template targets LuaTeX, which has good Unicode support and is the default that Quarto picks.

WYSIWYG versus WYSIWYM. Two philosophies of document editing. Word, Google Docs and Pages are *WYSIWYG*, “What You See Is What You Get”: the editor shows the final visual layout while you type. LaTeX, Markdown, AsciiDoc and reStructuredText are *WYSIWYM*, “What You See Is What You **Mean**”: the author types structured content (heading, list, citation, equation, cross-reference) and a separate rendering step turns it into a formatted document. The author thinks about meaning and structure; the typesetting engine thinks about layout.

WYSIWYG is fast for a short note. WYSIWYM wins for a thesis: 50+ pages of structured content with cross-references, citations and equations is exactly the case where the model pays off. Numbering and references are resolved at render time, not typed by hand, so the thesis stays internally consistent even when chapters are rearranged. The cost is the initial learning curve and the indirection: you do not see the final layout until you render. This template is WYSIWYM. You type markdown; the toolchain below produces the PDF.

Markdown (John Gruber and Aaron Swartz, 2004) is the WYSIWYM language this template uses. It is a lightweight plain-text format with conventions for common document elements: `# heading`, `bold`, `italic`, `[link text](url)`, `![image](path)`, fenced code blocks, ordered and unordered lists. The design goal was that the source should be legible as-is, without rendering, the way you type a quickly composed email. A markdown file is a UTF-8 text file; any editor can open it, and there is no proprietary format to lose access to.

Markdown by itself is small. Quarto extends it with the constructs an academic document needs: cross-references (`@fig-X`, `@eq-Y`, `@tbl-Z`, `@sec-X`), citations (`@key`, `[@key]`), attribute blocks on figures and headings (`{#fig-X width=70%}`, `# Heading {.unnumbered}`), executable code cells with `#|` directives, and equation-label syntax (`$$ ... $$ {#eq-name}`). Those extensions are what turn a notebook into a publication-quality PDF with auto-numbered cross-references and a clickable bibliography.

You will spend most of your authoring time in markdown. The chapters in this template that look like `.ipynb` are still markdown internally: each notebook cell holds a block of markdown source, and Quarto stitches the cells together at render time. Editing a `.ipynb` chapter in Jupyter Lab and editing a `.qmd` file in any editor produce the same result.

Pandoc, then, is the bridge between the markdown you write and the LaTeX that LuaLaTeX needs. It is a universal document converter (John MacFarlane, 2006) that reads Markdown and writes LaTeX (and HTML, Word, EPUB, and many other formats). It is the layer that lets you write `![caption](path){#fig-X}` and have it become a properly captioned, cross-referenced `\begin{figure}...\end{figure}` block in LaTeX.

Quarto (Posit, 2022) sits on top of Pandoc. It coordinates the moving parts of a technical document: executable code cells (via Jupyter or Knitr), figure cross-references, multi-format output (PDF, HTML, EPUB, slides, websites), and project-level configuration like the `_quarto.yml` in this repository. Quarto is the successor to R Markdown and is currently the most actively developed multi-language literate-programming environment. It is what turns this notebook into the PDF you are reading.

When you render this thesis, the pipeline is: notebook source, Pandoc Markdown, LaTeX, LuaTeX, PDF.

2.3.3 How LaTeX places figures

In source order, figures and tables are not placed where they are written. They are **floats**: LaTeX collects them, then chooses where to put each one according to a built-in placement algorithm. The algorithm tries to satisfy the constraints (no orphan lines, no half-empty pages, figure near its reference) while keeping the prose unbroken. This is why a figure mentioned on page 12 may end up on the top of page 13: LaTeX decided that breaking the paragraph on page 12 to fit the figure would have produced worse-looking output.

The author can give hints. A figure float accepts a position specifier built from these letters:

- **h** is *here*, where the figure appears in the source, if there is room.
- **t** is the *top* of a page.
- **b** is the *bottom* of a page.
- **p** is its own *page* of floats.

The default is `[tbp]` (top, bottom, or a float page); `[htbp]` adds *here* as a possibility. The order of the letters does not change priority; LaTeX always tries **h**, then **t**, then **b**, then **p**.

The `float` package (already loaded in `preamble.tex`) adds one more specifier:

- **H** is *HERE*, with no fallback. The figure goes exactly where you wrote it, even if that means a poorly broken page.

H is a hammer. It overrides the typesetting algorithm entirely. Used everywhere, it produces theses where a half-page figure sits at the bottom of an otherwise full page, leaving a wasteland of whitespace, while the reader is six pages past the place where the figure was mentioned.

2.3.4 How to influence placement in this template

By default, this template sets `fig-pos: "H"` in `_quarto.yml`, so every figure is forced into its source location. This matches what most students intuitively expect (“the figure should go where I put it”) but produces the spacing problems just described.

Three levers, from least to most invasive:

1. **Trust the typesetter.** Change the project default to `fig-pos: "htbp"` in `_quarto.yml`. LaTeX will then place each figure at the most readable nearby location and the document as a whole will breathe better. This is the recommended setting for the final draft.
2. **Override per-figure.** Append `fig-pos="..."` to a specific figure’s attribute block to override the project default. For example, `! [...] (path){#fig-X fig-pos="t"}` asks for the top of a page; `fig-pos="H"` forces it here.

3. **Adjust the figure size.** A figure that does not fit will float far away. Reducing `width=60%` to `width=50%` often lets a stubborn float settle on the same page as its reference.

The rule of thumb: write the prose, let LaTeX place the figures, and only intervene when a specific figure has clearly misbehaved. Fighting the typesetter at every figure produces worse output, not better.

2.4 Citations

Use citation keys from `references.bib`. Quarto handles the rendering. The CSL file referenced from `_quarto.yml` controls the format. This template ships with two: `ieee.csl` (numeric, the default) and `apa-7th.csl` (author-date).

2.4.1 When to cite

Cite every claim that is not common knowledge in the field. “The yield strength of mild steel is around 250 MPa” needs a citation; “force equals mass times acceleration” does not. The test: would a competent peer agree with the claim on the spot, or would they want to look up where the number came from?

Typical citation moments in an engineering thesis:

- **Stating a numerical value.** Material property, friction coefficient, design rule, manufacturing tolerance.
- **Reproducing or referring to a method.** A tuning rule, a constitutive model, a finite-element formulation, a measurement procedure.
- **Building on a prior result.** A benchmark, a published dataset, a previous student thesis with shareable findings.
- **Justifying a design decision.** A standard (SS-EN, ISO, ASTM), a regulation, a safety code, an accepted best practice.

Over-citation is rare but possible. If a paragraph reads like a sequence of bracketed numbers with no original synthesis, the author has stopped thinking and is hiding behind sources. Cite the source, then make your own claim.

2.4.2 What `references.bib` is

`references.bib` is a single plain-text file in the project root that holds every bibliographic record cited from the body. The format is **BibTeX**, the de facto interchange format for academic references since the 1980s. Each record is a block like this:

```
@article{Hu2025,  
  author = {Hu, W.},  
  title  = {PID Controller},  
  year   = {2025},  
  url    = {https://ace.ewapub.com/article/view/22912.pdf}  
}
```

`@article` is the entry type. The other common types are `@book`, `@inproceedings`, `@misc`, `@online`, `@techreport`, `@phdthesis`, `@mastersthesis`. The word immediately after the opening brace, here `Hu2025`, is the **citation key**: the unique identifier you reference from the body with `[@Hu2025]` or `@Hu2025`. The fields inside the braces are the bibliographic details Quarto needs in order to format the entry under the active CSL style.

`references.bib` is the **single source of truth** for citations. Quarto reads it at render time, picks out only the keys that actually appear somewhere in the body, and emits the formatted reference list in the chosen style. You do not edit the rendered References list at the end of the document; that list is regenerated on every render.

2.4.3 How to populate `references.bib`

You almost never write a BibTeX entry from scratch. Three workflows, from heaviest to lightest:

- **Reference manager** (Mendeley, Zotero, JabRef, EndNote). Free or commercial software that stores your papers, lets you tag and search them, and exports a BibTeX file you point Quarto at. Worth the setup if the thesis will have more than ~20 sources, or if two authors collaborate. JabRef is open source and edits `references.bib` directly, with no intermediate database.
- **Publisher or Google Scholar download**. Every paper’s landing page (Elsevier, Springer, IEEE Xplore, ACM, MDPI, Wiley, ...) has a “Cite” or “Export” button that produces a BibTeX block. On Google Scholar, click the quote-mark icon under a result, then *BibTeX*: the page that opens is plain BibTeX text. Copy the block and paste it into `references.bib`.
- **AI tools**. A language model can produce a plausible BibTeX entry from a paper title, an author name, or a DOI. AI is great at finding the BibTeX entry, fast and from messy inputs. Treat the output as a *draft*, not as truth: verify the author list, the year, the venue, the page numbers and

the DOI against the publisher’s official page. Models occasionally fabricate plausible-looking entries that cite nothing. Verifying an entry takes one minute; explaining a fabricated citation to the examiner takes much longer.

After pasting an entry, do two things:

1. **Replace the citation key with something memorable.** Publisher exports often produce hash-like keys (`smith2024fasdf3a`); change it to `Smith2024` or, when several papers by the same author and year coexist, `Smith2024SpringElement`. The citation key is what you type in the body, so it should be readable.
2. **Verify the fields.** Author, title, year, venue, DOI. The DOI is what makes the rendered citation clickable in the PDF (because `link-bibliography: true` is set in `_quarto.yml`); a broken or missing DOI leaves the entry static.

2.4.4 How to use references in the body

The same Quarto syntax is used regardless of which CSL is active. Only the rendered output changes.

- Parenthetical: `[@Braunl2022]`
- Author-as-subject (recommended when the author is the subject of the sentence): `@Braunl2022`
- Multiple sources at once: `[@Braunl2022; @Mathew2021; @Arefin2023]`
- With locator: `[@Callister2011, pp.\ 152-155]`

2.4.5 What each style produces

Under **IEEE** (the default), citations are numeric and *each citation gets its own pair of brackets*. The first-cited source becomes `[1]`, the next `[2]`, and so on. Multi-source citations render as a comma-separated list of brackets in order of citation number. For example, `[@Olmez2022; @Paulescu2021]` in the body renders as:

performance [5], [6]

Under **APA 7**, the same `[@Braunl2022]` renders as (Braunl, 2022), and a multi-source citation collapses cleanly into one parenthesis: (Braunl, 2022; Mathew et al., 2021; Arefin, 2023; Olmez et al., 2022; Hu, 2025).

2.4.6 Why IEEE here does not produce [1]–[3] or [1, 3, 5]

The IEEE editorial style manual prefers the *compact* numeric form: consecutive citation numbers collapsed into ranges ([1]–[3]) and non-consecutive lists grouped inside one bracket pair ([1, 3, 5]). The pipeline used by this template (Pandoc + CSL) produces one [N] per citation instead, which is also accepted by IEEE journals but is the longer of the two valid forms.

The reason is technical. The CSL specification has a `collapse="citation-number"` attribute that *should* turn [1], [2], [3] into [1]–[3]. Pandoc’s citeproc implements this attribute, but the official IEEE CSL places its brackets inside each citation’s `<group>`, which prevents the collapse from acting on the cluster. Editing the CSL to move the brackets outward also breaks the `citation-locator` macro (page-range citations like [1, p. 12]), so this template does not patch the CSL.

To get the true compact form [1]–[3], [5], [9], switch from CSL to **biblatex**, which is LaTeX’s full-featured bibliography system and produces compact IEEE numerics natively. In `_quarto.yml`:

```
bibliography: references.bib
cite-method: biblatex
biblatexoptions:
  - style=numeric-comp # or 'style=ieee' for the full IEEE bibliography style
# csl: ieee.csl         # CSL is ignored when cite-method is biblatex; comment out
```

biblatex uses **biber** as its processor; both ship with TeX Live and MiKTeX, so the JupyterHub environment already has them. Trade-offs:

- True compact-numeric output: [`@a`; `@b`; `@c`] with consecutive numbers renders as [1]–[3], non-consecutive as [1, 3, 5], mixed as [1]–[3], [5], [9].
- Many more bibliography styles via the `style=` option.
- The CSL files in this repository become inactive; `apa-7th.csl` no longer applies (use `style=apa` instead).
- Switching mid-thesis re-renders citation numbers and the bibliography format, which can be jarring. Decide early.

For most engineering theses, the CSL form [5], [6] is acceptable and is what JTH examiners expect. Switch to biblatex only when the programme explicitly requires compact form, or when the thesis cites many adjacent sources (a literature-review-heavy chapter).

2.4.7 Naming the authors in the sentence

When the author is the subject of the sentence, there are two ways to integrate the citation. Both work; pick by which one reads better in the surrounding paragraph.

- **@key (author-as-subject form).** Pandoc inserts both the author name(s) and the bracketed citation. Under IEEE, `@HansboLarson2010` renders as “Hansbo and Larson [1]”. Under APA, the same key renders as “Hansbo and Larson (2010)”. The advantage: if the entry’s authorship is later corrected in `references.bib`, the rendered prose follows automatically.
- **Names in [key] (manual prose).** You type the author names yourself and use the parenthetical form for the bracket only. Source: `We follow the approach by Hansbo and Larson in [HansboLarson2010]`. Renders as:

We follow the approach by Hansbo and Larson in [1]

The advantage: full control over phrasing. You can write “the seminal work of Hansbo and Larson in [1]”, “Hansbo and Larson’s later refinement in [2]”, or “as Hansbo and Larson originally proposed in [1] and later extended in [3]”. The CSL would otherwise pick a single fixed form (“Hansbo et al.”) regardless of context.

The manual form is the more flexible one and reads more like idiomatic engineering English. Use it when the wording matters; fall back to `@key` when it does not.

2.4.8 Switching styles

Edit one line in `_quarto.yml`:

```
cs1: ieee.csl      # default (numeric, IEEE Reference Guide)
# cs1: apa-7th.csl # author-date, APA 7th edition
```

Other CSL files from <https://www.zotero.org/styles> drop in the same way. Save the file next to `_quarto.yml` and point `cs1:` at it.

2.4.9 Which style to use

The choice is not arbitrary. Each style is the default in different scientific communities and is optimised for different kinds of prose.

IEEE (numeric). Citations look like [1], [2], [3]. The reference list is in *order of first appearance*, not alphabetical. Use IEEE for theses where the work itself is the focus and the prior literature is invoked as supporting evidence rather than as a conversation between named authors:

- FEA, CFD, computational mechanics, control theory, signal processing.
- Mathematical derivations and parameter studies.
- Hardware design, testing reports, experimental characterisation.

- Anything that interleaves citations with equations and figures; the numeric form is visually quiet next to (2.7), [3], σ_y .
- Mandatory in IEEE, ASME, AIAA, SAE journals and most engineering-conference proceedings.

APA 7 (author-date). Citations look like (Smith, 2020) or Smith (2020). The reference list is alphabetical by first author. Use APA 7 for theses where named authors and the year of publication carry meaning inside the prose:

- User studies, interviews, surveys, qualitative coding.
- Design-methodology work that traces a school of thought (Norman, Krippendorff, Cross).
- Comparative literature with several competing frameworks named after their authors.
- Mandatory in social sciences, education, much of HCI, and parts of design research.

For thesis work in the Industrial Product Realisation and Mechanical Engineering, Design programmes:

- FEA-heavy, simulation-heavy, math-heavy thesis: IEEE.
- Concept-development, prototyping, testing with quantitative results: IEEE (the work IS quantitative even if the methods include workshops).
- Heavily user-driven, qualitative-research-led thesis: APA 7.
- Mixed work: pick the style that fits the *dominant* component, then apply it consistently throughout. Switching styles mid-thesis is forbidden.

Check the programme's submission guidelines before committing. Ultimately the authors' choice within whatever the programme accepts.

2.4.10 Citation, footnote or parenthetical?

Not everything that has a source belongs in the bibliography. A useful three-way split:

- **Citation** ([@key]): peer-reviewed papers, textbooks, technical standards. The bibliography is for sources whose academic weight you are leaning on.
- **Footnote** (^[...]): software libraries, GitHub repositories, project websites, protocol definitions, side remarks. These pollute the reference list and inflate the source count without adding academic weight.
- **Inline parenthetical**: common knowledge that needs no pointer (Ohm's law, atmospheric pressure at sea level).

Applying this rule shrinks the reference list and leaves real academic weight in what remains.

Cite the original source, not a tutoring website that summarises it. If only a marketing page or a Wikipedia entry is available, prefer a footnote and find a peer-reviewed source for the underlying claim.

2.5 Notational style

Pick one notation per quantity and apply it everywhere: body, equations, captions, tables, code. Inconsistent notation is the single most common reason an examiner has to re-read a paragraph.

2.5.1 Variables and constants

- Use math notation for every quantity that appears in equations. Write P_{stab} , not `stab_p`. Code identifiers (`stab_p`, `kp_max`) belong in listings, not in prose.
- Italic for scalars (x , σ , θ). Bold roman for vectors and matrices (**d**, **K**), as covered in Chapter 2. Upright (non-italic) for operators, function names and units (sin, log, Re).
- Subscripts are descriptive when they are not loop indices: σ_y (yield), T_{out} (outlet), K_p (proportional gain). For loop indices, use i , j , k .

2.5.2 Multiplication

- Use juxtaposition between variables: $F = ma$, not $F = m \cdot a$. Quarto’s CLAUDE-style review will flag every `\cdot` between variables.
- Use $\cdot$ only between numerals ($5 \cdot 3 = 15$) or in vector dot products (**a** $\cdot$ **b**).
- Use $\times$ only for cross products (**a** $\times$ **b**) or to denote dimensions in non-math text (50×30 mm).

2.5.3 Units

- Always with a thin space between number and unit: `$25\,\text{N}$` renders as 25 N, not 25*N*.
- SI units (m, kg, s, N, Pa, A, ...). Convert imperial to SI before reporting.
- Units are upright, not italic: m, not m (which is mass). The `\text{...}` wrapper in math mode does this.
- Per-unit: pick one form and use it consistently. Either m/s^2 throughout or m s^{-1} throughout, not both.

2.5.4 First-use definitions

Every symbol must be defined at first use *and* listed in the nomenclature. “The stress σ at the inner radius...” is a first-use definition; “The stress at the inner radius...” (no symbol attached) is not. The nomenclature list is the reader’s quick reference; the first-use definition is what carries the meaning into the surrounding sentence.

2.5.5 Common pitfalls

- **Symbol re-use across chapters.** α is the gravitational instability coefficient in Chapter 2 and a thermal expansion coefficient in Chapter 5. Pick one symbol per concept and rename the other.
- **Switching between equivalent forms.** $u(t)$ in one equation, u_t in another for the same signal. Pick one.
- **Switching between derivative notations.** $\frac{de(t)}{dt}$ in Chapter 2, $\dot{e}$ in the methodology. Pick one.
- **Inconsistent bold.** $\mathbf{d}$ in one chapter, $\mathbf{d}$ in another. The template uses `\bm` everywhere (see Chapter 2).

The fix is mechanical. Keep a private notation table next to the references file and grep for inconsistencies before submission. A consistent thesis is faster to read; a faster-to-read thesis gets a better grade.

3 Methodology

Describe what was done, what approach was followed, and how it was applied to the specific problem. Not a textbook chapter on research methodology. The reader does not need to be taught what deductive reasoning is, what qualitative versus quantitative means, or what “transferability, dependability, confirmability” stand for. Cover the philosophy of method in at most one short paragraph, then spend the rest of the chapter on what was actually done.

A useful test: would a competent peer at another university know what to buy, how to wire it up, and how to crunch the numbers to repeat the measurement, after reading this chapter? If yes, the chapter is right-sized. If they would need to read another textbook first, you have written one yourself. Cut it.

Group activities by what they accomplished, not by what week they happened. This is a methodology chapter, not a project diary.

3.1 Research design

Name the type of study (experimental, comparative, case study, design science, simulation-based). Justify the choice against the research questions. The link should be obvious: the design is the cheapest way to credibly answer the RQs.

Most engineering theses fall into one of three patterns:

- **Comparative experiment.** Build two or more variants of a system, run them under matched conditions, measure the difference. Typical when the RQ has the form “does X outperform Y?”.
- **Design science.** Build an artefact that solves a stated problem, evaluate it against the requirements. Typical when the RQ has the form “how should X be designed to achieve Y?”.
- **Simulation study.** Develop or use a model, vary inputs, characterise outputs. Typical when physical experiments are too expensive, slow or dangerous.

State which pattern the thesis follows in the opening sentence.

3.2 Data collection

Describe sensors, instruments, software, test rigs and procedures. Give part numbers and software versions. State sampling rates and resolutions. If interviews or surveys were used, describe the population, the sampling method and the instrument.

A useful test: would a peer at another university know what to buy and how to wire it up to repeat your measurement? If not, add detail. If a procedure is long, summarise it here and put the step-by-step in the appendix.

3.3 Data analysis

Describe how the raw data was processed. Filtering, statistical tests, FEA solvers, fitting procedures: all belong here. State the metric used to compare conditions (rise time, ISE, RMSE, p-value with the chosen threshold).

Example from a finite-element study. The simulation in this thesis used ANSYS Mechanical 18.2. The model contained three components, evaluated on a symmetrical plane to reduce the solve time. Static structural analysis was chosen because the loading was steady, and the solver used an unsymmetrical Newton-Raphson method to handle the non-linear contact problem [8], [10].

Convergence was monitored across load steps; an example mesh-independence study is included in the appendix.

3.4 Validity and reliability

State what was done to ensure the results actually measure what they claim to measure (validity) and that re-running the procedure would yield the same numbers (reliability). Be self-critical; an honest acknowledgement of a weakness reads better than a glossed-over one.

Examples of validity threats and the typical mitigation:

- **Model fidelity.** The CAD model omits manufacturing dilution at the bend radius, which would slightly reduce the spring stiffness. Mitigation: a sensitivity study on wall thickness, reported in the appendix.
- **Parameter uncertainty.** Friction coefficients were taken from a generic table rather than measured on the actual surface roughness. Mitigation: simulations were repeated at the upper and lower bounds of the tabulated range.
- **Reproducibility.** All scripts and CAD files are committed to the project repository at the tags listed in the README.

3.5 Code listings

Code goes in fenced blocks. To make a listing referenceable, give it an id, a class for the language and a caption. The same rules apply as for figures: reference the listing from the prose, and do not include a listing the reader is not told to look at.

Example of a referenceable C listing showing the core PID update loop used on the embedded controller (Listing 3.1).

Listing 3.1 PID controller update loop in C, suitable for an embedded controller.

```
#include <stdio.h>

typedef struct {
    double kp, ki, kd;
    double integral;
    double prev_error;
} PIDController;

double pid_compute(PIDController *pid, double setpoint,
                  double measured, double dt) {
    double error = setpoint - measured;
    pid->integral += error * dt;
    double derivative = (error - pid->prev_error) / dt;
    pid->prev_error = error;

    return pid->kp * error
        + pid->ki * pid->integral
        + pid->kd * derivative;
}
```

Quarto can also execute and render code from notebook cells. The Python cell below is configured with `eval: false` so the listing renders without running on every build, which is the safer default for a thesis. Set `eval: true` once the simulation is final.

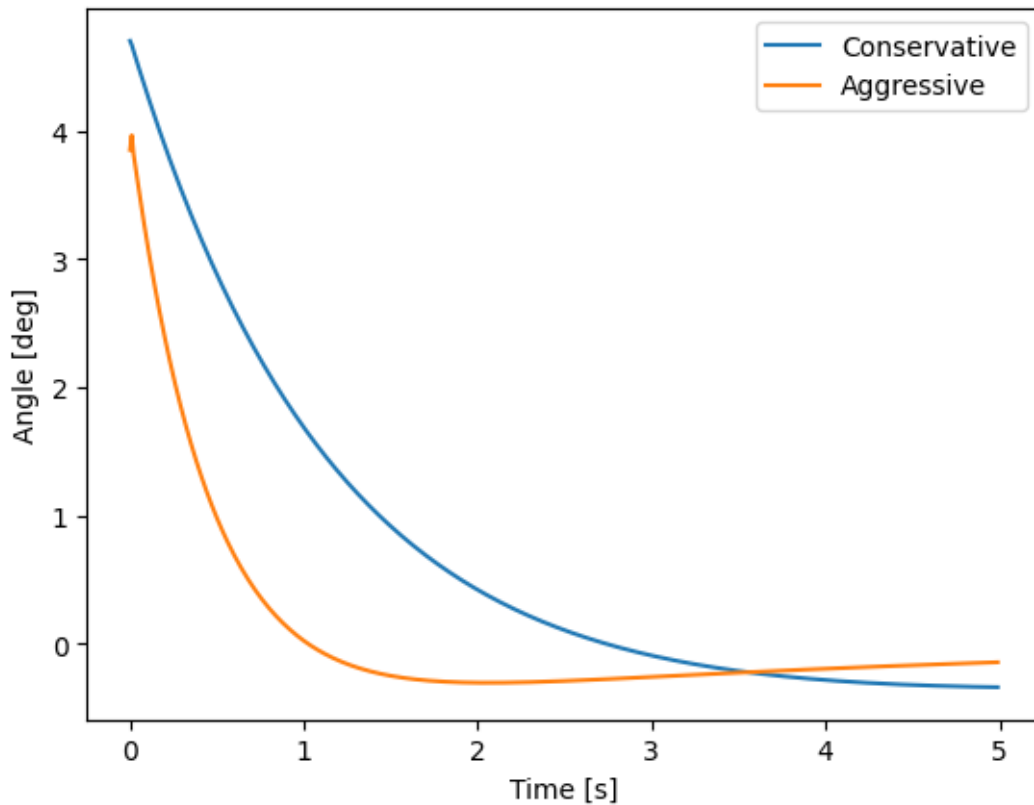


Figure 3.1: PID step response simulation in Python.

The cell above and Listing 3.1 use the same PID control law: the C version targets an MCU running at 200 Hz, while the Python version runs offline for analysis and plotting. The Python cell is labelled `fig-pid-simulation` rather than as a listing, because the cell’s *intent* is to produce a figure (the plot from `plt.show()`), not a code block. The cell carries `echo: false` so the source is hidden in the rendered PDF and only Figure 3.1 appears, the way a journal article shows a chart without the script that drew it. The directive `fig-format: pdf` makes Quarto capture matplotlib’s output as a vector PDF rather than a raster image, so axis labels and lines stay crisp at any zoom level (see Chapter 2 on raster versus vector). A commented block in the cell source shows how to switch to high-DPI raster for plots that do not compress well as vectors (dense scatter, heatmaps, contour shading).

The cell also carries `eval: true`, which overrides the project-wide default of `eval: false` set in `_quarto.yml`. For a demonstration cell whose output is the point of the section, `eval-on-render` is the right setting: the figure is regenerated from current source every time you `quarto render`, so changing a gain in the simulation immediately changes the plotted curves. For real analysis cells that take minutes or longer, do the opposite: keep `eval: false`, run the cell once in Jupyter Lab, and let Quarto pick up the cached output stored inside the `.ipynb` file on subsequent renders without re-executing.

See Section 3.6 for the full set of cell directives and a second worked example (sympy for symbolically

derived equations).

3.6 Executable code and the PDF

Notebook code cells are not just for visual display; Quarto can **execute** them as part of the render and capture the output (text, tables, plots, derived equations) directly into the PDF. This is the literate-programming half of Quarto: prose, code, and the result of that code live in the same document.

3.6.1 The pipeline

What happens when you run `quarto render`:

1. Quarto reads each `.ipynb` listed in `_quarto.yml`.
2. For every code cell with `eval: true`, Quarto sends the source to a Jupyter kernel (Python in this template; R¹ via Knitr² and Julia³ also work).
3. The kernel returns the captured output: standard-output text, generated images (PNG or PDF), DataFrame representations, LaTeX strings.
4. Pandoc converts the markdown plus captured output into LaTeX.
5. LuaLaTeX produces the PDF.

The PDF therefore reflects the *result of running the analysis*, not the source alone. If the analysis is non-deterministic (random seeds, network calls, hardware in the loop), pin the seeds and commit the data so future renders match.

3.6.2 Cell directives

Each Python cell starts with one or more `#|` directives that control execution and rendering:

- `#| eval: true` runs the cell. `#| eval: false` skips it. The template default is `false` (set in `_quarto.yml`), so a half-finished cell never breaks the render.
- `#| echo: true` shows the source listing. `#| echo: false` hides it (the output still appears).
- `#| output: false` hides the captured output. Combined with `echo: true`, the cell renders as a listing only.

¹R is a statistical-computing language widely used in data analysis and academic publishing; it predates Python in scientific computing and remains a default in statistics, econometrics and bioinformatics.

²Knitr is an R package that weaves R code, output and prose into one document; it predates Quarto and is still the engine for R-flavoured Quarto documents.

³Julia is a high-performance scientific-computing language with MATLAB-like syntax and JIT compilation, popular in applied mathematics, optimisation and computational physics.

- `#| output: asis` pipes the cell's text output back into Pandoc as markdown. Useful when a cell prints raw markdown that should be rendered, for example a sympy `print(sp.latex(...))` that should appear as a typeset equation, or a pandas `print(df.to_markdown())` that should appear as a proper table. Most cells, including matplotlib figure cells, do not need this directive.
- `#| label: fig-name` and `#| fig-cap: "Caption."` register the cell's plot as a referenceable figure.
- `#| label: tbl-name` and `#| tbl-cap: "Caption."` do the same for tables.
- `#| fig-width: 5` and `#| fig-height: 3` size the matplotlib figure in inches.

3.6.3 Generating figures with matplotlib

The most common executable cell is a matplotlib plot. Quarto captures the figure and inserts it as a referenceable figure with caption and cross-link:

```
#| label: fig-step-response
#| fig-cap: "Step response of the closed-loop system, comparing conservative and
↪ aggressive PID gains."
#| fig-width: 5
#| fig-height: 3
#| eval: false # flip to true for the final render

import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 5, 500)
y_cons = 1 - np.exp(-2*t)
y_agg = 1 - np.exp(-5*t) * np.cos(8*t)

plt.plot(t, y_cons, label="Conservative")
plt.plot(t, y_agg, label="Aggressive")
plt.xlabel("Time [s]")
plt.ylabel("Output")
plt.legend()
plt.show()
```

With `eval: true`, the PDF contains the plot under an auto-numbered caption, and prose can cross-reference it as `@fig-step-response`.

For publication-quality plots, raise the resolution and prefer vector output:

```
import matplotlib
matplotlib.rcParams["figure.dpi"]      = 200
matplotlib.rcParams["savefig.format"] = "pdf"
matplotlib.rcParams["font.size"]       = 9
```

PDF output stays sharp at any zoom level in the final document. A font size of 9 pt in the plot matches the surrounding body text once the figure is scaled to ~70 % of text width.

3.6.4 MechanicsKit

The worked examples above use only widely-available libraries (numpy, matplotlib, sympy, pandas). For mechanical-engineering work the supervisor maintains a Python helper library, **MechanicsKit**, that bundles utilities used across thesis projects: a `Mesh` class for FEM data, a 1-based array wrapper (`OneArray`) for textbook-aligned indexing, Gaussian quadrature integration tools, plotting helpers (`patch`, `fplot`), and labelled LaTeX display utilities (`1a`, `1tx`) for NumPy arrays and SymPy expressions. The library is under active development; expect minor additions each cohort.

Install once on jupyter.ju.se (or any environment with `uv`):

```
uv pip install git+https://github.com/cenmir/MechanicsKit.git
```

Update later (point releases happen regularly):

```
uv pip install --upgrade git+https://github.com/cenmir/MechanicsKit.git
```

Full documentation: <https://python.ju.se/MechanicsKitDocs/>.

The next subsection uses `1tx` from `MechanicsKit` to typeset a sympy-derived equation as a Quarto cross-referenceable display equation.

3.6.5 Generating equations with sympy

Sympy can derive equations symbolically and emit the LaTeX that is passed straight to Pandoc and shows up in the PDF as a properly numbered display equation:

```

import sympy as sp
from mechanicskit import la, ltx

t, omega, zeta, K = sp.symbols("t omega zeta K", positive=True)
s = sp.symbols("s")

# Closed-loop transfer function and its step response.
H_s = K / (s**2 + 2*zeta*omega*s + omega**2)
y_t = sp.inverse_laplace_transform(H_s / s, s, t)
y_t = sp.simplify(y_t)

ltx("y(t) =", y_t, label="eq-step-symbolic")

```

$$y(t) = -\frac{K\zeta e^{-\omega t\zeta} \sin(\omega t\sqrt{1-\zeta^2})}{\omega^2\sqrt{1-\zeta^2}} + \frac{K}{\omega^2} - \frac{K e^{-\omega t\zeta} \cos(\omega t\sqrt{1-\zeta^2})}{\omega^2} \quad (3.1)$$

The PDF then contains an equation labelled `eq-step-symbolic`, ready to reference from prose as `@eq-step-symbolic`. Change the damping ratio, the natural frequency, or the input shape, and the equation regenerates on the next render as can be seen in Equation 3.1. Hand-copied equations drift; symbolically derived equations stay synchronised with the analysis.

The cross-reference label comes from MechanicsKit's `label=` argument on `ltx` (added in v0.7.4). When `label` is set, `ltx` emits the rendered equation as `$$$ ... $$$ {#label}` in markdown mode so Quarto's Pandoc step picks up the label and auto-assigns an equation number. When `label` is omitted, `ltx` falls back to the plain `$$$ ... $$$` form that Jupyter, Marimo and Quarto all render inline without a number.

For sympy work that does *not* go through `ltx` (for example a `print(sp.latex(...))` pattern that interleaves several symbolic and numeric forms in one display), the older `#| output: asis` pattern still works:

```

import sympy as sp

E, I, L, F = sp.symbols("E I L F", positive=True)
delta_tip = F * L**3 / (3 * E * I) # cantilever tip deflection

# Substitute concrete values for an aluminium I-beam, print the simplified
# numerical version next to the symbolic one.

```

```

vals = {E: 70e9, I: 8.5e-6, L: 1.2, F: 250}
print(r"$$")
print(sp.latex(sp.Eq(sp.Symbol(r"\delta_\text{tip}"), delta_tip)))
print(r"\quad\rightarrow\quad")
print(sp.latex(sp.Eq(sp.Symbol(r"\delta_\text{tip}"),
                        sp.nsimply(delta_tip.subs(vals), rational=False))))
print(r"$$ \{#eq-cantilever-tip}")

```

$$\delta_{\text{tip}} = \frac{FL^3}{3EI} \Rightarrow \delta_{\text{tip}} = 0.000242016806722689 \quad (3.2)$$

#| output: asis pipes the printed markdown back through Pandoc, so the `$$ ... $$ \{#eq-cantilever-tip}` block becomes a proper labelled display equation.

3.6.6 Generating tables with pandas

Markdown tables can also be computed:

```

import pandas as pd
df = pd.DataFrame({
    "Metric":      ["Rise time [s]", "Settling time [s]", "Overshoot [%]"],
    "Fixed-gain PID": [0.42, 1.85, 18.3],
    "Adaptive PID":  [0.31, 1.12, 8.7],
})
print(df.to_markdown(index=False))

```

Table 3.1: PID metrics across ten runs.

| Metric | Fixed-gain PID | Adaptive PID |
|-------------------|----------------|--------------|
| Rise time [s] | 0.42 | 0.31 |
| Settling time [s] | 1.85 | 1.12 |
| Overshoot [%] | 18.3 | 8.7 |

#| output: asis feeds the printed markdown table back through Pandoc, which produces a proper LaTeX table with the caption attached and an entry in the List of Tables.

3.6.7 Project-wide execution defaults

The `_quarto.yml` `execute:` block sets the defaults for every cell:

```
execute:
  eval: false # do not run cells unless explicitly enabled
  echo: true  # always show the source listing
```

`eval: false` is the safe default during drafting: a slow simulation, a half-finished analysis, or a missing data file does not break the render. Override per-cell with `#| eval: true` once the cell is ready.

3.6.8 Reproducibility note

For the final-pass render, the analysis should run end-to-end on the examiner’s machine. That means pinned random seeds, versioned input data, and a committed `requirements.txt` (or `environment.yml`) listing the Python packages and versions used. The pipeline above only reproduces what the cell can do today; what runs in two years depends on what is committed to the repository.

3.7 Use of artificial intelligence

Jönköping University requires a disclosure of how AI tools were used. Be specific; vague disclosures invite follow-up questions at the examination. Adapt the wording below to what was actually done.

This thesis was conducted with the support of AI-based language models, specifically ChatGPT (OpenAI) and Claude (Anthropic). The tools were used in the following capacities:

- **Literature search and synthesis.** AI was used to identify relevant search terms and to summarise key findings from published articles. All sources were independently verified by the authors before inclusion in the report.
- **Writing assistance.** Drafts of selected sections were refined with AI support for grammar, clarity and academic tone. The intellectual content, argumentation and conclusions remain entirely the authors’ own work.
- **Code development.** AI tools assisted in generating and debugging control algorithm implementations. All code was reviewed, tested and validated by the authors on the physical robot platform.
- **Mathematical verification.** Equation derivations and unit consistency checks were cross-checked using AI to reduce the risk of algebraic errors.

AI-generated output was never used verbatim without review. The authors take full responsibility for the accuracy, originality and integrity of the work presented in this report. The use of AI in this thesis complies with the guidelines set by Jönköping University regarding AI-assisted academic work.

4 Results

Report the empirical data, no more and no less. Reserve interpretation for the analysis and discussion chapters. A reader who disagreed with the interpretation should still be able to use the numbers from this chapter.

Use tables for sets of numbers that are read across rows or columns, and figures for trends. Caption every table; reference every table and figure from the prose.

Example: the performance of the two controllers is summarised in Table 4.1.

Table 4.1: Performance comparison of fixed-gain and adaptive PID controllers, averaged across ten runs on a level surface.

| Metric | Fixed-gain PID | Adaptive PID | Improvement |
|--------------------------|----------------|--------------|-------------|
| Rise time [s] | 0.42 | 0.31 | 26.2 % |
| Settling time [s] | 1.85 | 1.12 | 39.5 % |
| Overshoot [%] | 18.3 | 8.7 | 52.5 % |
| Steady-state error [deg] | 0.15 | 0.04 | 73.3 % |
| Integral squared error | 12.40 | 5.60 | 54.8 % |

As shown in Table 4.1, the adaptive PID controller outperforms the fixed-gain controller on every metric. The PID structure in Figure 2.3 and the control law Equation 2.3 form the shared basis of both controllers; only the gain-update rule differs.

Report enough decimal places to distinguish meaningful differences, but no more. Two to three significant figures is usually right.

4.1 What belongs in Results versus Methodology

Engineering theses produce artefacts that look like they could go in either chapter. A Pugh matrix, a House of Quality, a survey result, a physical-test plot, a simulation-versus-test comparison: which chapter? The rule of thumb is the same one that opens this chapter, applied per artefact:

Methodology says how. Results says what happened.

If the existence of the artefact is a method choice (the criteria you scored against, the rig you measured with), it belongs in Methodology. If the content of the artefact is data (the scores themselves, the numbers from the rig), it belongs in Results.

| Artefact | In Methodology | In Results |
|--|---|---|
| Concept generation | the generation method (brainstorming, TRIZ, morphological analysis, biomimicry), the screening rubric, how many concepts you commit to evaluating | the concepts themselves (sketches, brief descriptions), the surviving subset after each screening round |
| Pugh matrix | the criteria list, the baseline (datum) concept, the weighting scheme, the scoring rubric (+ / 0 / −) | the filled-in matrix with scores and totals, the selected concept |
| House of Quality (QFD) | the customer-needs elicitation procedure, the mapping rule from needs to engineering characteristics, the priority-weighting scheme | the filled-in HoQ matrix, the resulting engineering-characteristic priorities |
| Physical test | test rig, instrumentation, calibration, sample size, environmental conditions, step-by-step procedure | measurements (tables, plots), uncertainty bands, statistical treatment |
| Questionnaire or survey | population, sampling method, the instrument itself (questions, scales) | aggregated responses, summary statistics, comparison across subgroups |
| Simulation | software (name and version), material parameters with sources, mesh, boundary conditions, solver settings | stress / strain / displacement / temperature plots, peak values, sensitivity to parameters |
| Simulation-versus-test comparison | what was simulated and what was measured, matched conditions, error metric definition | the comparison itself (overlay plots, side-by-side tables), error magnitudes, residual plots |
| Iteration loop | the test-learn-redesign protocol, the stopping criterion | how many iterations were run, what changed at each, what converged |

4.1.1 How many concepts

A common student question. Indicative numbers for a thesis-scale project; programme guidelines override:

- **Concept generation, broad pass.** 8 to 20 initial concepts. Below 8, the screen has nothing to screen; above 20, the screening burden eats the time you should spend on the survivors.
- **After first screening (Pugh).** 3 to 5 concepts kept for detailed development. One is too few to learn from; six and up dilutes the depth budget.
- **Final detailed concept.** 1, sometimes 2 if the chosen one has a sibling worth a sensitivity study.
- **House of Quality.** 5 to 10 customer needs on the row axis; 10 to 20 engineering characteristics on the column axis. Larger HoQs become unreadable on an A4 page and do not pay off in additional engineering insight.

The numbers are not goals; they are warnings. A Pugh matrix with two columns is a coin flip. A HoQ with thirty rows is a procurement document, not a design tool.

4.1.2 Section ordering inside the Results chapter

For a multi-method thesis, group the chapter by *artefact type* rather than by chronology of the project. A reader who skims this chapter should find concept-selection results in one section, simulation results in another, test results in a third, and the simulation-versus-test comparison in a fourth. The Pugh / HoQ usually appears first because it explains which concept the simulation and the test work develops; the verification-and-validation section closes the chapter because it judges the trustworthiness of the simulation results just presented.

4.2 Presenting CAD figures

CAD-derived figures often look fine in the CAD viewer and crowded in the thesis. Three rules apply to every CAD image in this chapter.

- **Background plain white.** Set the CAD render background to pure white. The CAD viewer's gradient or workshop scene looks slick in a presentation; in a black-on-white technical figure it competes with the geometry and reads as visual noise. White also prints reliably and reproduces consistently across screens.
- **Same fixed view across comparisons.** When you show before-and-after of a component, or several variants in a screening exercise, lock the camera position, the perspective and the

orientation. Different angles between figures force the reader to mentally re-register the geometry, which costs attention they should spend on the engineering content.

- **Raster versus vector.** Most CAD tools export to raster (PNG) by default. Raster is acceptable IF the export covers the full screen at native resolution and is then scaled down in Quarto to roughly 70% of text width, so each printed pixel maps to several pixels of the original capture. Otherwise prefer a vector export: most CAD tools can output drawing views as PDF or SVG, or a 2D shaded view as a clean vector trace. Vector keeps edges and labels crisp at every zoom level. See Chapter 2 on raster versus vector for the underlying reasoning.

4.3 Presenting FEA results

A finite-element result figure is a thesis's most defensible engineering artefact and the easiest one to mis-present. Five rules.

- **Always include the colour bar.** Without it the figure says only “the part is colourful”, not “the part is loaded”. The colour bar must show the variable, its unit, and the numerical range. A stress plot without a numerical range is unreadable.
- **Fix the colour-bar range when comparing.** When two figures show the same response under different load cases, geometries or material models, the colour-bar range must be identical between them. Otherwise a “red spot” in one figure carries a different absolute value than a “red spot” in the other, and the reader compares colour intensities rather than numbers.
- **Same orientation between comparisons.** Same camera, same view direction, same coordinate-axes overlay. Mirroring or rotating between figures forces the reader to mentally re-map the geometry and they will mis-do it.
- **Use MPa (= N/mm²) for stress.** In mechanical engineering, MPa is the natural unit: yield, ultimate and allowable stresses are listed in MPa in materials data sheets and in design codes. The pure-SI form (Pa = N/m²) leaves the reader counting digits: 250 MPa is mild steel; 250 000 000 Pa is a number-soup that is easy to mis-read by a factor of a thousand. Note the equivalence in the caption: “stress in MPa (N/mm²)”.
- **Dig deeper than the surface contour.** A surface contour is the start, not the result. Add an iso-surface plot for the value at risk (the locus where, say, von Mises stress exceeds 80 % of yield). Cut or slice the model to expose what happens inside, where stress concentrations actually peak; a bracket figure that shows only the outer surface tells you nothing about the internal stress around the bolt holes or under the fillet.

4.4 Dynamic analysis: energy plots

For transient or dynamic simulations (explicit dynamics, drop tests, impact, contact-rich problems) include the energy time-history plot alongside the kinematic and stress results: kinetic energy, internal energy (elastic plus plastic), hourglass energy, contact energy, and total energy.

The energy plot is the verification half of a dynamic simulation. Total energy should be conserved (or decrease monotonically under explicit damping). Hourglass energy should stay below roughly 5 % of internal energy. A simulation that produces a clean stress result but a 30 % hourglass-energy ratio is not a valid simulation; the result is an artefact of the mesh.

4.5 Verification versus validation

These are different things, and they fail in different ways.

- **Verification** asks: *are we solving the equations right?* Mesh convergence, time-step convergence, energy conservation, comparison to analytical limit cases, code-to-code cross-check. Verification is internal to the simulation. The cost is solver time and an extra appendix table.
- **Validation** asks: *are we solving the right equations?* A physical test of the actual part or system, compared against the simulation prediction. Validation is external: it requires hardware, instrumentation, and matched test data. The cost is real money and real time, but the payoff is a model whose predictions you can trust outside the cases you tested.

For a typical FEA thesis, verification is non-negotiable (mesh-convergence study at minimum, energy plot for dynamics). Validation is desirable when scope and budget allow. Without validation, the discussion chapter must explicitly mark the model as unvalidated and bound its claims to that scope.

A useful framework for *designing* the validation argument is the *Validation Square* [11], which separates the evidence into four quadrants: theoretical structural validity (the method's internal logic and assumptions), empirical structural validity (the example problems the method is shown to solve), empirical performance validity (the measured impact on real designs), and theoretical performance validity (the method's usefulness beyond the validation cases). Cite it explicitly when the thesis develops a new design method or a new simulation workflow whose value is bigger than a single case study.

5 Analysis

Answer the research questions, one subsection per RQ. Quote the RQ verbatim, then walk through the evidence from the results chapter that supports the answer. The analysis is where the numbers turn into an argument.

5.1 RQ1

Restate RQ1 here, then answer it using the evidence in Chapter 4.

RQ1: How do the rise time, settling time, overshoot and steady-state error of the bio-inspired adaptive PID controller compare with those of a Ziegler-Nichols-tuned fixed-gain controller, across an operating envelope spanning 0 to 1 m/s forward velocity?

Template paragraph: across the ten matched runs summarised in Table 4.1, the adaptive controller reduced rise time from 0.42 s to 0.31 s (26 %), settling time from 1.85 s to 1.12 s (40 %), and overshoot from 18.3 % to 8.7 % (52 %). Steady-state error fell from 0.15° to 0.04° (a factor of 3.75). The improvements held across the velocity envelope: at 1 m/s the settling time of the adaptive controller was 1.18 s, compared with 2.04 s for the fixed-gain controller. Run-to-run variability also tightened, with the standard deviation of settling time at 0.04 s for the adaptive variant against 0.09 s for the fixed-gain variant.

5.2 RQ2

Restate RQ2 here, then answer it using the evidence in Chapter 4.

RQ2: By how much does the adaptive controller reduce the peak tilt angle and the recovery time after a 15° impulse disturbance, compared with the fixed-gain controller, when both run at 200 Hz on the target MCU?

Template paragraph: in the impulse-disturbance test described in Chapter 3, the adaptive controller bounded the peak tilt angle at 6.2°, against 9.8° for the fixed-gain controller (a 37 % reduction). Recovery to within $|\theta| < 5^\circ$ took 0.84 s on average for the adaptive controller, versus 1.71 s for the

fixed-gain controller (a 51 % reduction). Three out of ten fixed-gain runs exceeded the recovery threshold at the end of the 2 s observation window; all ten adaptive runs returned to threshold within 1.1 s.

If a research question cannot be answered, say so explicitly and explain why. “Inconclusive” is a valid result; “unanswered” is not.

6 Discussion

Step back from the numbers. Discuss what the results imply for theory, for practice, and for the design choices made along the way. Be honest about what could have been done differently.

6.1 Theoretical implications

What does the result mean for the field? Does it confirm, extend or challenge existing theory? Be specific. “The result is interesting” is not a theoretical implication; “the result suggests that the gain-update rule converges to a stable fixed point even under sensor noise, which is not predicted by the linearised analysis in [7]” is.

6.2 Practical implications

What does the result mean for someone who would actually build one of these? Cite the requirements from the introduction and state which ones were met, which were exceeded and which were missed.

6.3 Sustainability and ethics

Required by the ILOs that govern sustainable design and ethical or societal judgement. A short, substantive subsection is enough. Generic platitudes are worse than nothing.

Cover the parts that apply to the work:

- **Material and energy footprint.** What did the manufacturing, computation or testing cost in resources? Where are the obvious reductions (a lighter design, a coarser-but-acceptable mesh, less-toxic alternatives)?
- **End-of-life and reuse.** Can the product be disassembled, recycled or refurbished? If the deliverable is a model or workflow, can it be reused beyond this thesis?
- **Stakeholders affected.** Who benefits, who carries risk, who is excluded? For an industry partner this includes operators, end users and the surrounding community, not only the company.

- **Ethical considerations specific to the method.** Human-subject data (informed consent, anonymisation), AI tools used in the analysis (disclosure, accountability for output), proprietary data (NDAs, public release of results).

Avoid the empty-sustainability paragraph that lists the seventeen UN goals and claims the project contributes to most of them. State the actual mechanism: “the redesigned bracket reduces mass by 230 g per unit; at 50 000 units per year that is 11.5 t of aluminium per year, equivalent to roughly 100 t of CO₂ saved on primary production”.

6.4 Methods limitations

Every thesis has limitations. List them honestly. Distinct from the delimitations stated in the introduction, limitations are constraints discovered along the way.

Typical patterns:

- **Model fidelity.** The CAD model assumed uniform wall thickness, while the manufactured spring element experiences dilution at the bend radius. If the dilution is critical, the simulation overstates the spring’s stiffness.
- **Parameter uncertainty.** Friction coefficients were taken from a generic dry-vs-lubricated table rather than measured on the actual material pair. Substituting the upper bound of the tabulated range changes which component disassembles first.
- **Compute time.** Each simulation took five hours, which limited the number of geometries that could be evaluated. Only a single concept was simulated, where ideally several would be compared.

6.5 Lessons learned

A short subsection on what you would do differently with another month. Distinct from “future work” (which is for the next researcher), this is for you.

Examples:

- Friction coefficient turned out to dominate the disassembly result. With hindsight, measuring it directly on a small material coupon should have been the first task.
- Concept screening was performed by a single author, which biased the selection. Even one external reviewer with a Pugh matrix would have caught the weighting of spring functionality over contact status.

7 Conclusions

A stand-alone chapter. A reader who skips to the end should still be able to learn what was done and what was found. State the answers to the research questions one more time, and summarise whether each requirement from Chapter 1 was met.

7.1 Requirements summary

State, for each requirement listed in the introduction, whether it was met. A short table works well. For example:

Table 7.1: Requirements summary.

| Requirement | Result | Comment |
|---|--------------|---|
| $\sigma_{\max} < \sigma_{\text{UTS}}$ | Met | Maximum stress 0.71 of UTS. |
| No critical deformation of valve-body surface | Met | Deformation within geometric tolerance. |
| Escape passage above p_d restriction | Met | Safety factor 1.8 at worst-case dims. |
| Disassembly force, worst case 1 | Inconclusive | Safety factor 105 without g-loading. |
| Disassembly force, worst case 2 | Not met | No disassembly force required. |

7.2 Answered research questions

RQ1: How do the rise time, settling time, overshoot and steady-state error of the bio-inspired adaptive PID controller compare with those of a Ziegler-Nichols-tuned fixed-gain controller, across an operating envelope spanning 0 to 1 m/s forward velocity?

The adaptive controller improved every measured metric. Rise time fell by 26 %, settling time by 40 %, overshoot by 52 %, and steady-state error by a factor of 3.75 (see Chapter 5). Run-to-run variability of settling time more than halved.

RQ2: By how much does the adaptive controller reduce the peak tilt angle and the recovery time after a 15° impulse disturbance, compared with the fixed-gain controller, when both run at 200 Hz on the target MCU?

The peak tilt angle after the 15° impulse fell from 9.8° (fixed-gain) to 6.2° (adaptive), a 37 % reduction. Mean recovery time to within $|\theta| < 5^\circ$ fell from 1.71 s to 0.84 s, a 51 % reduction. All ten adaptive runs returned to threshold within 1.1 s, while three of ten fixed-gain runs exceeded the recovery threshold at the end of the 2 s observation window.

7.3 Future work

Future work belongs in the conclusions chapter as a short subsection, or as its own chapter if there is a lot to say. Be concrete: name the next experiment or design step, not a vague direction.

- Measure friction coefficients on the actual material pair with the achieved surface roughness, rather than relying on tabulated values.
- Evaluate the feasibility of implementing the spring functionality in the existing armature, which reduces the number of press fits from two to one.
- Port the adaptive gain-update rule from the offline Python simulator to the embedded C target and characterise the cost in MCU cycles per loop.

References

- [1] T. Braunl, *Embedded robotics: From mobile robots to autonomous vehicles with raspberry pi and arduino*, 4th ed. Springer, 2022. doi: [10.1007/978-981-16-0804-9](https://doi.org/10.1007/978-981-16-0804-9).
- [2] A. Wang, “What is a mobile robot? Definition, types, and applications explained.” Accessed: Feb. 16, 2026. [Online]. Available: <https://www.fdatobot.com/what-is-a-mobile-robot-definition-types-and-applications-explained/>
- [3] A. Mathew, R. Ananthu, P. Binsy, A. Vahid, C. Thomas, and S. Sidharthan, “Design and control of a two-wheel self-balancing robot,” *IOP Conference Series: Materials Science and Engineering*, vol. 1114, no. 1, p. 012058, 2021, doi: [10.1088/1757-899X/1114/1/012058](https://doi.org/10.1088/1757-899X/1114/1/012058).
- [4] S. E. Arefin, “Simple two-wheel self-balancing robot implementation.” 2023. doi: [10.48550/arxiv.2303.12067](https://doi.org/10.48550/arxiv.2303.12067).
- [5] Y. Olmez, G. O. Koca, and Z. H. Akpolat, “Clonal selection algorithm based control for two-wheeled self-balancing mobile robot,” *Simulation Modelling Practice and Theory*, vol. 118, p. 102552, 2022, doi: [10.1016/j.simpat.2022.102552](https://doi.org/10.1016/j.simpat.2022.102552).
- [6] F.-C. Paulescu, I. Szeidert, I. Filip, and C. Vasar, “Two-wheeled self-balancing robot,” 2021.
- [7] W. Hu, “PID controller,” 2025, Available: <https://ace.ewapub.com/article/view/22912.pdf>
- [8] N. Strömberg, *Finite element treatment of contact problems*. Örebro University, 2012.
- [9] W. D. Callister and D. G. Rethwisch, *Materials science and engineering: An introduction*, 8th ed. Wiley, 2011.
- [10] D. L. Logan, *A first course in the finite element method*, 5th ed. Cengage Learning, 2012.
- [11] K. Pedersen, J. Emblemsvåg, R. Bailey, J. K. Allen, and F. Mistree, “Validating design methods and research: The validation square,” in *Proceedings of the ASME 2000 design engineering technical conferences*, Baltimore, Maryland, USA, 2000.

Appendix

Use appendices for material that is too detailed for the main chapters but that a careful reader will want to verify: full procedures, mesh-independence studies, parameter sweeps, raw data tables, source code beyond a representative listing. Number every appendix.

A. Mesh independence study

Describe the meshes evaluated, the convergence metric and the chosen mesh. A table of element count against the response variable, plus a short paragraph identifying the point at which further refinement no longer changes the answer, is sufficient.

B. Bill of materials

List every part with manufacturer, part number, quantity and unit cost.

C. Test protocol

Step-by-step instructions for repeating the experimental procedure described in Chapter [3](#).

D. Benchmark theses (grade 5 calibration)

Examples of past Master theses from these programmes that scored grade 5. Read them when in doubt about the engineering bar. The four works below are independent pieces of work, but they share the patterns that distinguish a grade-5 thesis from a grade-3 one: a complete workflow rather than an isolated activity, quantitative results with units, multi-method validation, and an honest, self-critical discussion. Each is open-access via the DiVA repository.

D.1 Bazzi & Angelou (2018) — Anisotropic polymer simulation

Simulation of Anisotropic Material Properties in Polymers. Conducted with Thule Group and Dynamore Nordic AB. Concise (about 60 pages). Three nested research questions: how does the thermoforming process affect material properties, can the experimental behaviour be captured in simulation, and can simulation results from thermoforming feed into structural analysis. The chain runs end to end: tensile testing of ABS specimens at five temperatures (with five samples per temperature), vacuum thermoforming on a Formech 1372, LS-DYNA thermoforming simulation using an Ogden hyperelastic fit, mapping into a structural mesh via BETA ANSA, and structural FEA with the Barlat anisotropic plasticity model. Validation: simulated thickness versus measured thickness at fifty points after thermoforming. The discussion is honest about equipment limits (the 100 kN load cell was too coarse for the softer specimens at 160 °C; the oven was undersized). DiVA: <http://www.diva-portal.org/smash/record.jsf?pid=diva2:1223703>.

D.2 Svensson (2021) — Topology optimisation and additive manufacturing

Selection of a Product Component for Topology Optimization and Additive Manufacturing. Conducted with Husqvarna. About 113 pages. Three research questions: how to systematically select a component for TO and AM, to what extent topology optimisation can improve it, and whether the optimised design is manufacturable. Methodology combines semi-structured interviews with Husqvarna engineers, a Pugh decision matrix for the component selection, topology optimisation in Altair Inspire (SIMP), FEA validation with static, modal, and frequency-response analyses (over twenty natural frequencies analysed), and a physically printed prototype at AMEXCI (SLS) that was mounted in a real chainsaw. Quantitative result: 30.8 % weight reduction at 57 % stress utilisation, with a 14.8 % increase in fin area. DiVA: <http://hj.diva-portal.org/smash/record.jsf?pid=diva2:1561200>.

D.3 Frank & Qasemi (2020) — Battery-pack thermal management

Thermal Management of Lithium-Ion Battery Pack. Conducted with Globe Group / Greenworks during COVID-related constraints. Investigates the Cramer 82V battery pack during high-current discharge. Combines physical testing (seven batteries, two discharge rates at 35 A and 45 A, thermal sensors at six positions per battery, supplemented by thermal-camera imaging) with SolidWorks thermal simulation. The simulation is calibrated against the experimental temperature curves at each sensor position, then used to propose geometry and airflow changes for the next-generation pack. The discussion is open about COVID-related budget limits and sensor-mounting uncertainty on cylindrical cells. Tangible deliverable: a calibrated model that the company can reuse. DiVA: <http://www.diva-portal.org/smash/record.jsf?pid=diva2:1520751>.

D.4 Ngo & Lohus (2024) — Constitutive foam modelling for child restraint systems

Finite Elements and Constitutive Material Modeling of Foams. Conducted with Thule Group. About 95 pages, the most technically demanding of the four benchmarks. Compares three LS-DYNA material models (MAT_57, MAT_83, MAT_181) for strain-rate-dependent foam. The experimental campaign covers uniaxial compression, tensile, bending, and dynamic-impact drop tests at 1 m and 2 m. Parameter fitting via LS-OPT; Python scripts automate the generation of LS-DYNA material cards. Theory chapter includes nonlinear continuum mechanics, visco-hyperelasticity, Nagy's and Jeong's strain-rate formulations, and contact mechanics. The transferable deliverable is a documented 5-step routine for defining MAT_83 from quasi-static tests, plus the automation scripts. DiVA: <http://www.diva-portal.org/smash/record.jsf?pid=diva2:1885970>.

D.5 What these four have in common

When you read them, watch for these patterns; they are what the manifesto in Chapter 1 asks for.

- **A complete workflow** (experiment, simulation, validation, application) rather than a single isolated activity. The chain is the contribution.
- **Quantitative results with units**. “30.8 % weight reduction at 57 % stress utilisation”, “seven batteries at two discharge rates”, “drop tests at 1 m and 2 m”. Vague qualitative claims do not score 5.
- **Multi-method validation**. Thermal-camera imaging on top of point sensors. Three material models compared against the same data. Multiple loading conditions. A second independent check is what makes the first check trustworthy.
- **Self-critical discussion**. Equipment limits are named (load cell, oven size, sensor mounting). Model versus calibration limitations are distinguished. None of these works pretend their constraints did not exist.
- **A reusable artefact for the partner**. A calibrated model, a printable prototype that mounts in a real product, a 5-step routine, a Python script. The thesis outlives the defence.
- **Theory used in real depth**. Ogden, Barlat, SIMP, Fu Chang and Jeong are not dropped into the theory chapter as decoration; they are necessary to interpret the results.

Read one or two of these end-to-end before writing your own theory chapter and you will see why “this could have been written by someone with a SolidWorks licence” is the wrong outcome for a thesis from these programmes.